

CSE 256 | Lecture 10: Transformers (part 2)

Ndapa Nakashole

February 5, 2026

Administrative Matters

- ▶ **Programming Assignments (PAs):** Please submit **typed solutions** rather than handwritten ones - legible and easy to grade.
- ▶ Hand-drawn figures are OK when they are **conceptual**, but not for graphs or tables
 - ◉ Well-designed figures are an important part of scientific communication. Tools: Matplotlib, Tikz, or even PowerPoint are encouraged.

Recap: Self-Attention for an Entire Sequence

1 Score (relevance)

$$\mathbf{S} = \frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} \in \mathbb{R}^{|Q| \times |K|}$$

2 Normalize

$$\mathbf{A} = \text{softmax}(\mathbf{S}) \in \mathbb{R}^{|Q| \times |K|}$$

3 Aggregate (weighted sum)

$$\mathbf{Z} = \mathbf{A}\mathbf{V} \in \mathbb{R}^{|Q| \times d_v}$$

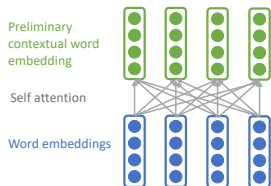
Each output vector is a relevance-weighted combination of values, computed independently for every query.

Recap: Stacked Transformer Blocks

► Multi-Head Self-Attention

- Aggregates information from different representation subspaces in parallel

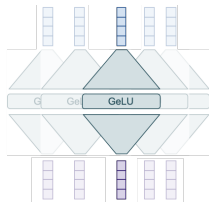
Attention is the "communication" stage



► Position-wise Feedforward Networks

- Two linear transformations with an activation g in between.
- $$\text{FFN}(x) = W_2(\text{GeLU}(W_1x + b_1)) + b_2$$

Feedforward is the "think" stage



Today

Transformers continued

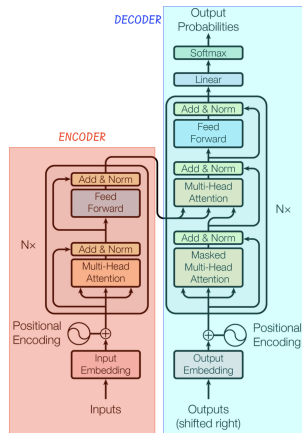
1 Attention

- Self-attention
- Multiple attention heads in parallel
- Interspersing of multi-head attention and feedforward layers
- Stacking of Transformer blocks
- Decoder self-attention (masked)
- Encoder-decoder attention (cross-attention)

2 Positional Encoding

3 Design Choices for Stable Training

- Residual connections (ResNet structure)
- Layer normalization
- Scaled dot-product attention



Transformer Encoder Invariants

The d_{model} Representation Space

Sequence length L ; model dimensionality d_{model}

- ▶ Token embeddings live in the shared model space:

$$E_{\text{token}} \in \mathbb{R}^{|\mathcal{V}| \times d_{\text{model}}}$$

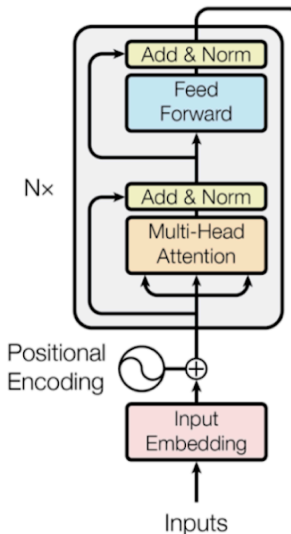
- ▶ Hidden representations at every layer:

$$\mathbf{H}^{(\ell)} \in \mathbb{R}^{L \times d_{\text{model}}}$$

- ▶ Attention projects *out of* this space:

$$\mathbf{Q} \in \mathbb{R}^{L \times d_k} = \mathbf{H} \mathbf{W}_q, \quad \mathbf{W}_q \in \mathbb{R}^{d_{\text{model}} \times d_k}$$

Main Invariant of the Transformer Encoder



Main invariant: Attention and FF layers project *out of* $\mathbb{R}^{d_{\text{model}}}$, all encoder blocks ultimately project *back into* $\mathbb{R}^{d_{\text{model}}}$

- Allows block stacking without shape mismatches

Projecting Back to d_{model}

- ▶ After attention, we obtain: $Z \in \mathbb{R}^{L \times d_v}$
- ▶ But Transformer is designed so **all hidden states live in** $\mathbb{R}^{d_{\text{model}}}$
- ▶ Thus, Self-attention has a final output projection

$$H' = ZW_o$$

$$W_o \in \mathbb{R}^{d_v \times d_{\text{model}}}$$

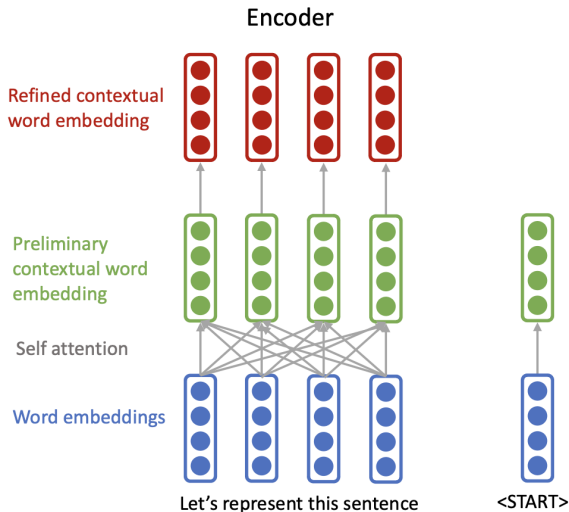
- ▶ The FFN preserves shape:

$$H \in \mathbb{R}^{L \times d_{\text{model}}}$$

Decoding with the Transformer

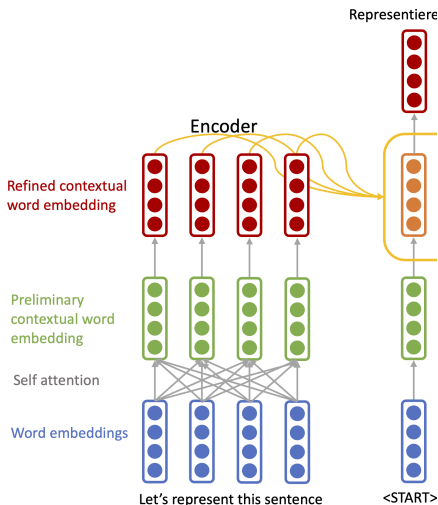
Decoding at Inference Time (1/n)

Generate one token at a time



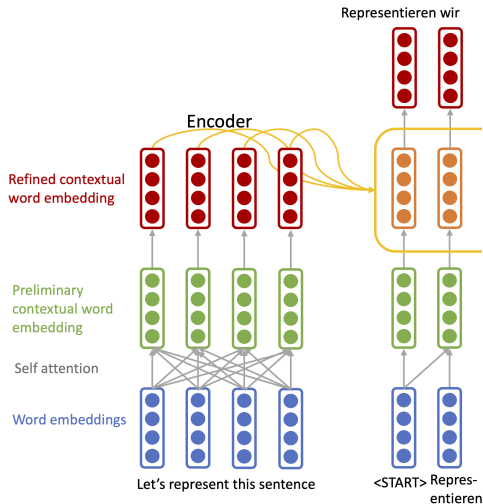
Decoding at Inference Time (2/n)

one token at a time



- Decoder (green) provides the queries; encoder (red) provides the keys and values.

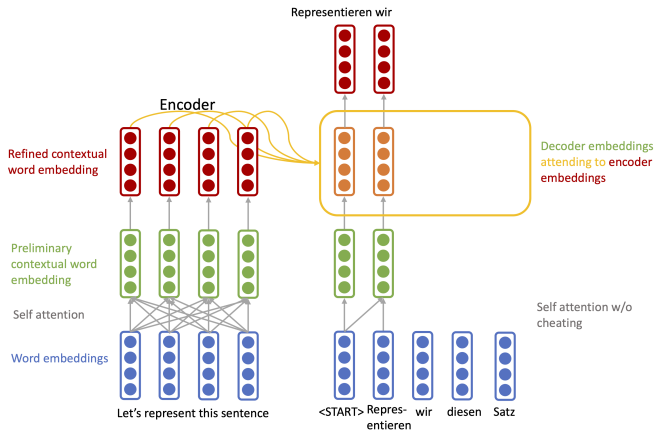
Decoding at Inference Time (3/3)



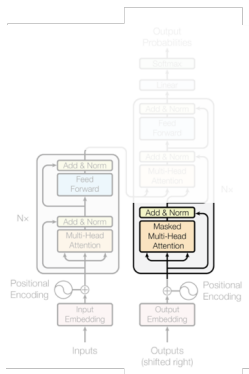
- Decoder conditions on all **previously generated tokens**, and the encoder output.

Decoding at Training Time: Teacher Forcing

- ▶ During training, full target sequence is available
- ▶ No need to wait for predictions from previous time steps
 - ◉ Instead, condition on the **ground-truth** previous tokens
 - ◉ This is called **teacher forcing**



Decoding at Training Time: Masked Self-Attention



At training time: Masked self-attention

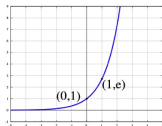
- If we had to train on single word at a time, training would **SLOW**. Want to process **entire sequence in parallel**.
- How? Mask out the future tokens in the input sequence. Set attention scores to $-\infty$ for illegal connections, thus zero attention prob.

At generation time:

- There is no such trick. Must generate one token at a time. Hence, autoregressive decoding is slow.

Attention Masking Details

- ▶ We want to mask with upper triangular matrix. In PyTorch use: `torch.triu`
- ▶ Setting **attention score** to $-\infty$ ensures that attention probability will be zero.



$$e_{ij} = \begin{cases} q_i^\top k_j, & j \leq i \\ -\infty, & j > i \end{cases}$$

We can look at these
(not greyed out) words

	[START]	The	chef	who
[START]		$-\infty$	$-\infty$	$-\infty$
The			$-\infty$	$-\infty$
chef				$-\infty$
who				

Masked Self-Attention: Summary

- ▶ Modify self-attention so certain positions are *inaccessible*
- ▶ **Mechanism:** add a mask to the attention scores

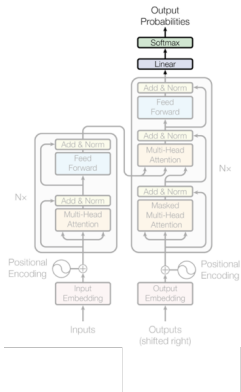
$$QK^{\top} \longrightarrow QK^{\top} + M$$

- ▶ The mask M contains:
 - ◉ 0 for allowed positions
 - ◉ $-\infty$ for disallowed (future) positions

$$\text{softmax} \left(\frac{QK^{\top} + M}{\sqrt{d_k}} \right) V$$

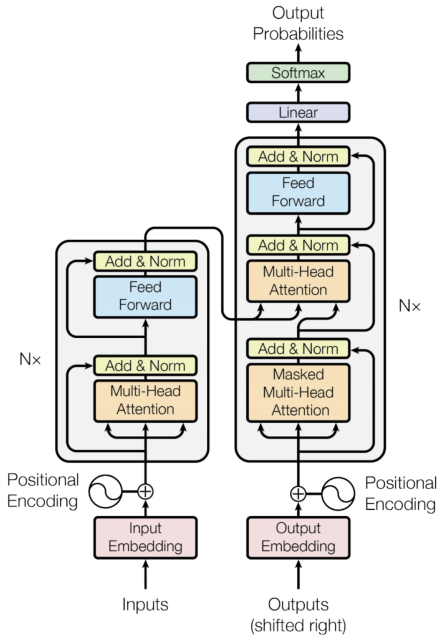
The Output Layer and Full Picture

Output Layer

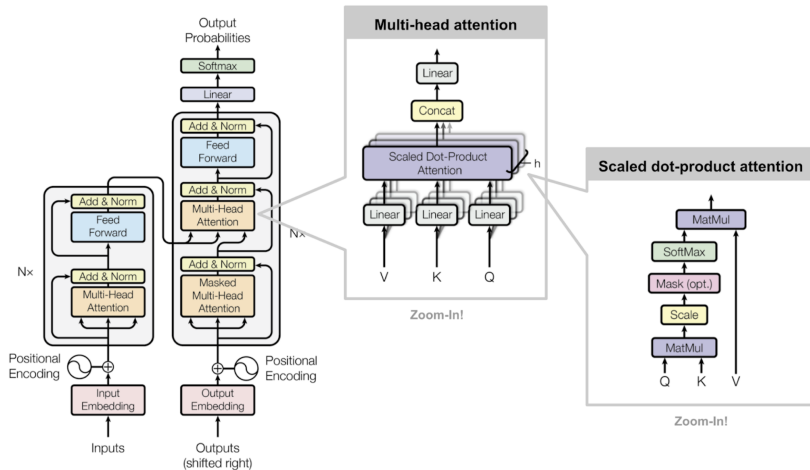


- ▶ The output layer converts the decoder output into a probability distribution over the vocabulary.
 - ◉ The output layer is a linear layer with a softmax activation.
- ▶ (Greedy decoding): The word with the highest probability is the predicted next word in the output sequence.

Full Transformer Architecture for Seq2Seq Tasks



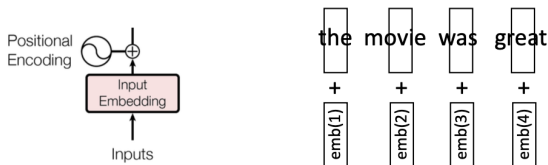
Transformer Architecture: Full Picture



Positional Encoding

Problem: No Notion of Word Order in Attention

"the cat sat on the mat"



- ▶ Consider representing each **sequence index** as a **vector** $p_i \in \mathbb{R}^d$, for $i \in \{1, 2, \dots, n\}$ are position vectors
- ▶ The positioned embedding is:

$$\tilde{x}_i = x_i + p_i$$

Sometimes used in practice (also in PA2)! But **can't extrapolate** to indices outside n used during training. Also adds **extra parameters** in matrix $p \in \mathbb{R}^{d \times n}$

Using Trigonometric Functions for Positional Encoding

- Vaswani et al 2017: Calculate each dimension with sines/cosines of different frequencies

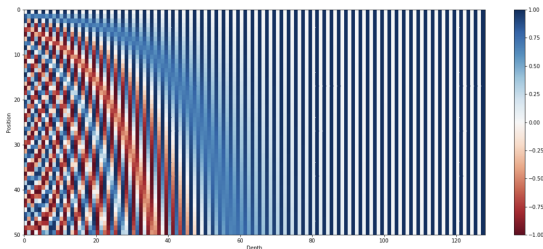


Figure 2 - The 128-dimensional positional encoding for a sentence with the maximum length of 50. Each row represents the embedding vector $\vec{p}_i$

Why? closer words get higher dot products

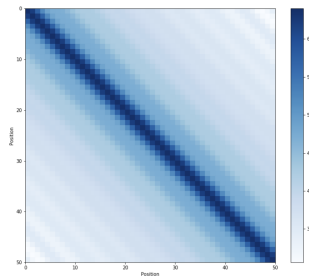


Figure 3 - Dot product of position embeddings for all time-steps

Absolute vs Relative Positional Encodings

- ▶ **Absolute:** add an encoding to the input in **hope** that relative position will be captured (Vaswani et al. 2017)
 - ⊙ Not learned, fixed functions (sines/cosines)
 - ⊙ Fails to extrapolate to longer sequences as periods repeat
- ▶ **Relative: Explicitly** encode relative position e.g., "I" is 3 words away from "run"
 - ⊙ Press et al. 2021: "Attention with Linear Biases" (ALiBi), **parameter-free** linear bias added to dot-product similarity. Extrapolates to longer sequences.

The diagram shows the combination of absolute and relative positional encodings. On the left is a 5x5 grid representing absolute encodings, where the value at row i and column j is $q_i \cdot k_j$. The grid is lower triangular, with the main diagonal highlighted in a darker blue. To the right of this grid is a plus sign (+). Next is another 5x5 grid representing relative encodings, where the value at row i and column j is $r_{ij} = j - i$. This grid is also lower triangular, with the main diagonal highlighted in a darker blue. To the right of this second grid is a dot followed by the variable m ($\cdot m$), indicating that the sum of the two grids is scaled by m .

$q_1 \cdot k_1$				
$q_2 \cdot k_1$	$q_2 \cdot k_2$			
$q_3 \cdot k_1$	$q_3 \cdot k_2$	$q_3 \cdot k_3$		
$q_4 \cdot k_1$	$q_4 \cdot k_2$	$q_4 \cdot k_3$	$q_4 \cdot k_4$	
$q_5 \cdot k_1$	$q_5 \cdot k_2$	$q_5 \cdot k_3$	$q_5 \cdot k_4$	$q_5 \cdot k_5$

+

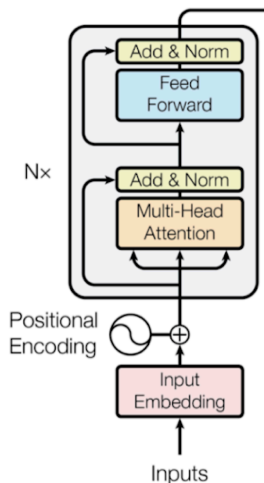
0				
-1	0			
-2	-1	0		
-3	-2	-1	0	
-4	-3	-2	-1	0

$\cdot m$

Design Choices for Stable Training

Residual Connections and Layer Normalization

Residual Connections



- Instead of learning a full transformation

$$\mathbf{y} = F(\mathbf{x})$$

the model learns a **residual**:

$$\mathbf{y} = \mathbf{x} + F(\mathbf{x})$$

- Interpretation:
 - ◉ “Start from $\mathbf{x}$ ”
 - ◉ “Only change what needs changing”
- If $F(\mathbf{x}) \approx 0$, the layer behaves like an identity map

Residual Connections Help Training



- ▶ Deep networks are hard to train because gradients vanish or explode
- ▶ Residual connections provide:
 - ▶ a direct path for gradients
- ▶ In Transformers:
 - ◉ attention blocks refine representations
 - ◉ feedforward blocks refine them further
 - ◉ residuals ensure nothing essential is lost

Residual Connections Help Gradients Flow

Consider a residual block:

$$\mathbf{y} = \mathbf{x} + F(\mathbf{x})$$

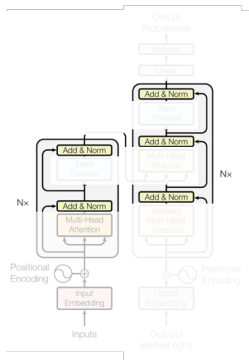
During backpropagation:

$$\frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \mathbf{I} + \frac{\partial F(\mathbf{x})}{\partial \mathbf{x}}$$

- ▶ The identity term $\mathbf{I}$ provides a **direct gradient path**
- ▶ Gradients can flow even if $\frac{\partial F}{\partial \mathbf{x}}$ is small
- ▶ Early layers keep receiving learning signal

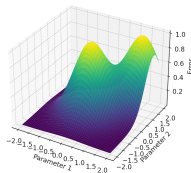
Layer Normalization

Helps stabilize training by normalizing activations

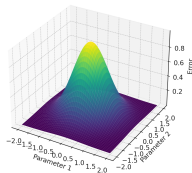


Layer normalization: Normalizes the outputs to be within a consistent range, preventing too much variance in scale of outputs

Error Surface Before Normalization



Error Surface After Normalization



Layer Normalization: What Is Being Normalized?

- ▶ LayerNorm is applied **independently to each token**
- ▶ For a hidden representation at one position:

$$\mathbf{h}_i \in \mathbb{R}^{d_{\text{model}}}$$

- ▶ Normalization is performed **across features**, not across tokens
 - ◉ Each token is normalized on its own feature vector

Layer Normalization: The Computation

Given a token representation:

$$\mathbf{h} = (h_1, \dots, h_d) \in \mathbb{R}^d$$

Compute statistics *within the vector* mean and standard deviation:

$$\mu = \frac{1}{d} \sum_{j=1}^d h_j, \quad \sigma = \sqrt{\frac{1}{d} \sum_{j=1}^d (h_j - \mu)^2}$$

Normalize and re-scale:

$$\text{LayerNorm}(\mathbf{h}) = \gamma \odot \frac{\mathbf{h} - \mu}{\sigma} + \beta$$

- ▶ $\gamma, \beta \in \mathbb{R}^d$ are learned vectors (gain, bias - can undo normalization if needed)
- ▶ Mean 0, variance 1 before re-scaling

Layer Normalization: Worked Example

Consider a token representation:

$$\mathbf{h} = [2, 4, 6], \quad d = 3$$

Step 1: Compute mean

$$\mu = \frac{1}{3}(2 + 4 + 6) = 4$$

Step 2: Center the vector

$$\mathbf{h} - \mu = [-2, 0, 2]$$

Step 3: Compute standard deviation

$$\sigma = \sqrt{\frac{1}{3}(4 + 0 + 4)} = \sqrt{\frac{8}{3}} \approx 1.63$$

Step 4: Normalize

$$\frac{\mathbf{h} - \mu}{\sigma} \approx [-1.23, 0, 1.23]$$

Result: mean 0, variance 1 (before learned re-scaling)

Pre-Norm vs Post-Norm Transformers

Why Pre-Norm Is Critical for Deep Models

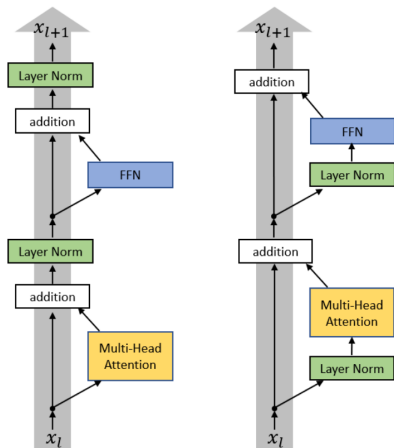


Figure from Xiong 2020

- ▶ **Post-Norm (original Transformer, BERT):** LayerNorm applied *after* the residual addition
- ▶ **Pre-Norm (current LMs):** LayerNorm applied *before* the sublayer

Pre-Norm:

- ▶ Keeps the main residual path **close to the identity**.
- ▶ Better **gradient flow, stable training** at large depth

Great Results with Transformers

Transformer Results on Machine Translation

First, Machine Translation from the original Transformers paper!

Table 2: The Transformer achieves better BLEU scores than previous state-of-the-art models on the English-to-German and English-to-French newstest2014 tests at a fraction of the training cost.

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	41.29	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	$3.3 \cdot 10^{18}$	
Transformer (big)	28.4	41.8	$2.3 \cdot 10^{19}$	

10^{21} is 100 times slower than 10^{19} . If something takes 40 days in the 10^{21} regime, it would take 0.4 days (or about 9.6 hours) in the 10^{19} regime.

Transformer Results: Document Generation

WikiSum dataset*

Next, Document generation

Model	Test perplexity	ROUGE-L
<i>seq2seq-attention, $L = 500$</i>	5.04952	12.7
<i>Transformer-ED, $L = 500$</i>	2.46645	34.2
<i>Transformer-D, $L = 4000$</i>	2.22216	33.6
<i>Transformer-DMCA, no MoE-layer, $L = 11000$</i>	2.05159	36.2
<i>Transformer-DMCA, MoE-128, $L = 11000$</i>	1.92871	37.9
<i>Transformer-DMCA, MoE-256, $L = 7500$</i>	1.90325	38.8

The old standard

Transformers all the way down.

Transformer Results: Pretraining and Fine-tuning

Parallel training in Transformers allows for efficient pretraining on large corpora, leading to powerful language models

On this popular aggregate benchmark, for example:



All top models are Transformer (and pretraining)-based.

Rank	Rank Name	Model	URL	Score
1	DeBERTa Team - Microsoft	DeBERTa / TuringNLRv4	🔗	90.8
2	HFL iFLYTEK	MacALBERT + DKM		90.7
+	3 Alibab DAMO NLP	StructBERT + TAPT	🔗	90.6
+	4 PING-AN Omni-Sinitic	ALBERT + DAAF + NAS		90.6
	5 ERNIE Team - Baidu	ERNIE	🔗	90.4
	6 T5 Team - Google	T5	🔗	90.3

► More next week when we talk about pretraining!

*[Liu et al., 2018]

What happens with Multi-Head Attention?

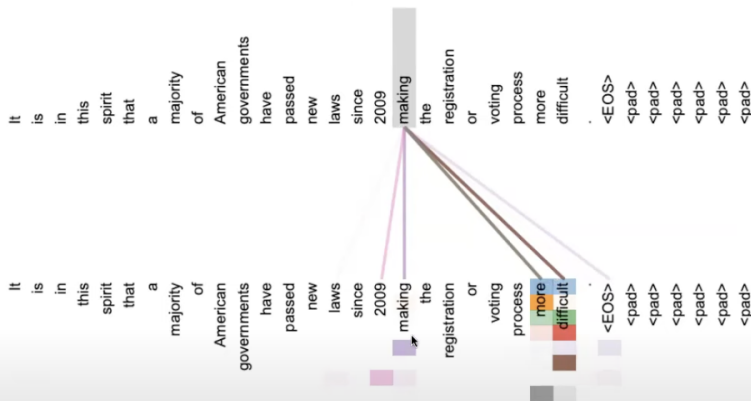


Figure 3: An example of the attention mechanism following long-distance dependencies in the encoder self-attention in layer 5 of 6. Many of the attention heads attend to a distant dependency of the verb 'making', completing the phrase 'making...more difficult'. Attentions here shown only for the word 'making'. Different colors represent different heads. Best viewed in color.

Do Transformer Modifications Improve Performance?

“Surprisingly, we find that most modifications do not meaningfully improve performance.”

Model	Params	Ops	Step/s	Early loss	Final loss	SGLUE	XSum	WebQ	WMT EnDe
Vanilla Transformer	223M	11.1T	3.50	2.182 ± 0.005	1.838	71.66	17.78	23.02	26.62
GeLU	223M	11.1T	3.58	2.179 ± 0.003	1.838	75.79	17.86	25.13	26.47
Swish	223M	11.1T	3.62	2.186 ± 0.003	1.847	73.77	17.74	24.34	26.75
ELU	223M	11.1T	3.56	2.270 ± 0.007	1.932	67.83	16.73	23.02	26.08
GLU	223M	11.1T	3.59	2.174 ± 0.003	1.814	74.20	17.42	24.34	27.12
GeGLU	223M	11.1T	3.55	2.130 ± 0.006	1.792	75.96	18.27	24.87	26.87
ReLU	223M	11.1T	3.57	2.145 ± 0.004	1.803	76.17	18.36	24.87	27.02
SoLU	223M	11.1T	3.55	2.315 ± 0.004	1.948	68.76	16.76	22.75	25.99
SwiGLU	223M	11.1T	3.53	2.127 ± 0.003	1.789	76.00	18.20	24.34	27.02
LiGLU	223M	11.1T	3.59	2.149 ± 0.005	1.798	75.34	17.97	24.34	26.53
Sigmoid	223M	11.1T	3.63	2.291 ± 0.019	1.867	74.31	17.51	23.02	26.30
Softplus	223M	11.1T	3.47	2.207 ± 0.011	1.850	72.45	17.65	24.34	26.89
RMS Norm	223M	11.1T	3.68	2.167 ± 0.008	1.821	75.45	17.94	24.07	27.14
ResZero	223M	11.1T	3.51	2.262 ± 0.003	1.939	61.69	15.64	20.90	26.37
ResZero + LayerNorm	223M	11.1T	3.26	2.223 ± 0.006	1.858	70.42	17.58	23.02	26.29
ResZero + RMS Norm	223M	11.1T	3.34	2.221 ± 0.009	1.875	70.33	17.32	23.02	26.19
Fixup	223M	11.1T	2.95	2.382 ± 0.012	2.067	58.56	14.42	23.02	26.31
24 layers, $d_g = 1536, H = 6$	224M	11.1T	3.33	2.200 ± 0.007	1.843	74.89	17.75	25.13	26.89
18 layers, $d_g = 2048, H = 8$	223M	11.1T	3.38	2.185 ± 0.005	1.831	76.45	16.83	24.34	27.10
8 layers, $d_g = 4096, H = 18$	223M	11.1T	3.69	2.190 ± 0.005	1.847	74.58	17.69	23.28	26.85
6 layers, $d_g = 6144, H = 24$	223M	11.1T	3.70	2.201 ± 0.010	1.857	73.55	17.59	24.60	26.66
Block sharing	65M	11.1T	3.91	2.497 ± 0.037	2.164	64.50	14.53	21.96	25.48
+ Factorized embeddings	45M	9.4T	4.21	2.631 ± 0.305	2.183	60.84	14.00	19.84	25.27
+ Factorized & shared embeddings	20M	9.1T	4.37	2.907 ± 0.313	2.385	53.95	11.37	19.84	25.19
Encoder only block sharing	170M	11.1T	3.68	2.298 ± 0.023	1.929	69.60	16.23	23.02	26.23
Decoder only block sharing	144M	11.1T	3.70	2.352 ± 0.029	2.082	67.93	16.13	23.81	26.08
Factorized Embedding	227M	9.4T	3.80	2.208 ± 0.006	1.855	70.41	15.92	22.75	26.50
Factorized & shared embeddings	202M	9.1T	3.92	2.320 ± 0.010	1.952	68.69	16.33	22.22	26.44
Tied encoder/decoder input embeddings	248M	11.1T	3.55	2.192 ± 0.002	1.840	71.70	17.72	24.34	26.49
Tied decoder input and output embeddings	248M	11.1T	3.57	2.187 ± 0.007	1.827	74.86	17.74	24.87	26.67
Untied embeddings	273M	11.1T	3.53	2.195 ± 0.005	1.834	72.99	17.58	23.28	26.48
Adaptive input embeddings	204M	9.2T	3.55	2.250 ± 0.002	1.899	66.57	16.21	24.07	26.66
Adaptive softmax	204M	9.2T	3.60	2.364 ± 0.005	1.982	72.91	16.67	21.16	25.56
Adaptive softmax without projection	223M	10.8T	3.43	2.229 ± 0.009	1.914	71.82	17.10	23.02	25.72
Mixture of softmaxes	232M	16.3T	2.24	2.227 ± 0.017	1.821	76.77	17.62	22.75	26.82
Transparent attention	223M	11.1T	3.33	2.181 ± 0.014	1.874	54.31	10.40	21.16	26.80
Dynamic convolution	257M	11.8T	2.65	2.403 ± 0.009	2.047	58.30	12.67	21.16	17.03
Lightweight convolution	224M	10.4T	4.07	2.370 ± 0.010	1.989	63.07	14.86	23.02	24.73
Evolved Transformer	217M	9.9T	3.09	2.220 ± 0.003	1.863	73.67	10.76	24.07	26.58
Synthesizer (dense)	224M	11.4T	3.47	2.334 ± 0.021	1.962	61.03	14.27	16.14	26.63
Synthesizer (dense plus)	243M	12.6T	3.22	2.191 ± 0.010	1.840	73.98	16.96	23.81	26.71
Synthesizer (dense plus alphas)	243M	12.6T	3.01	2.180 ± 0.007	1.828	74.25	17.02	23.28	26.61

Do Transformer Modifications Transfer Across Implementations and Applications?

Sharan Narang [*]	Hyung Won Chung	Yi Tay	William Fedus
Thibault Evrly [†]	Michael Matena [‡]	Karishma Malkan [†]	Noah Fiedel
Noam Shazeer	Zhenzhong Lan [†]	Yanqi Zhou	Wei Li
Nan Ding	Jake Marcus	Adam Roberts	Colin Raffel [†]

Long Contexts: Why Recurrence Is Reappearing

Self-attention

- ▶ Computes *all pairwise interactions*
- ▶ Quadratic cost in sequence length

$$\mathcal{O}(L^2)$$

Recurrent models

- ▶ Process sequences step by step
- ▶ Linear cost in sequence length

$$\mathcal{O}(L)$$

Implications:

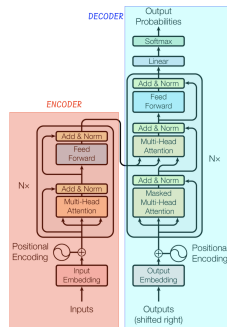
- ▶ For *very long contexts*, quadratic attention becomes costly
- ▶ This has renewed interest in modern recurrent architectures

Current landscape

- ▶ Transformers dominate for moderate context lengths
- ▶ New models (e.g., RWKV, Mamba) revisit recurrence
- ▶ Goal: combine *long-context efficiency* with strong representations

The Transformer: versatile architecture

Domain	Data Type
Natural Languages	Text
Programming Languages	Code
Computer Vision	Images
Speech	Audio
Proteins Folding	Protein Sequences
Music	Audio
Video	Video
...	...



That's it!

- ▶ Transformers are going to be the foundation for the much of the rest of this class and are a ubiquitous architecture nowadays
- ▶ Many details to get right, many ways to tweak and extend them, but core idea is the multi-head self attention and their ability to contextualize items in sequences